Benchmarking and Exploring the Capabilities of LLMs for Attack Investigations

Aniket Anand
University of Chicago
USA

Yiwei Hou
University of California, Berkeley
USA

Daniel Fields
University of Chicago
USA

Alex Kantchelian
Google
USA

David Tao
Google
USA

Kurt Thomas
Google
USA

Grant Ho
University of Chicago
USA

arXiv:2606.10281v2 [cs.CR] 26 Sep 2026

Abstract

This paper presents AuditBench, a new benchmark dataset for evaluating the capabilities of LLMs at investigating security-related system audit logs. We design and use this benchmark to explore the performance of LLMs on four log-investigation tasks that incident response teams commonly perform, ranging from triaging alerts generated by detectors to identifying persistence mechanisms on compromised systems. AuditBench consists of system audit logs collected from Linux and Windows machines, and spans over 50 different security investigation scenarios, including both malicious and benign activity. Using our benchmark, we evaluate and analyze the performance of five frontier LLMs at analyzing audit logs for attack investigations. Our analysis illuminates how LLM performance and error profiles vary according to different design choices, such as differences in model size, data representation, prompt construction, and specific investigation tasks. Additionally, we characterize the quality of the explanations produced by LLMs and the types of errors that models make across our benchmark. Collectively, our work provides a foundation for assessing the capabilities of LLMs for investigating security logs, novel insights for practitioners using LLMs in security operations, and important directions for future research.

CCS Concepts

• **Security and privacy → Intrusion detection systems**.

Keywords

Audit logs, security investigation, benchmark, incident response

1 Introduction

Many organizations employ dedicated security teams with expertise in analyzing network and system audit logs to detect, investigate, and recover from attacks. Unfortunately, security teams suffer from an overwhelming volume of false alarms, resulting in slow remediation times and missed attacks [2, 55, 60]. Analyzing even a small number of alerts and their corresponding log data requires specialized knowledge and remains a challenging process. In particular, this forensic work requires analysts to not only reason about large amounts of data, but also bridge the semantic gap between low-level details reported by audit logs (e.g., system calls) and the higher-level activity that describes attackers' concrete behavior and outcomes.

The emergence of large language models (LLMs) has led to significant interest and the development of many LLM-driven services for security forensics and log analysis [8, 26, 45]. Despite this growing deployment and papers exploring various security applications of LLMs [63], limited published work has explored the capabilities of LLMs for security audit log analysis and attack investigations. This gap exists because of a lack of sufficiently diverse and well-labeled data for such evaluations, and the broad and amorphous nature of real-world attack investigations.

To address this problem, we develop and present a new evaluation benchmark, **AuditBench**, that assesses the capabilities of LLMs for analyzing audit logs relevant to security investigations. Our benchmark spans four common security investigation tasks, with labeled log datasets spanning 51 distinct attack and benign scenarios. We construct this dataset by combining both (1) newly-generated audit log data that captures a variety of benign behavior and dozens of simulated attack behaviors from the MITRE ATT&CK framework [12], and (2) carefully curated and labeled scenarios from the largest public dataset of system audit logs (DARPA OpTC [15]). As part of this benchmark, we develop an extensible and automated evaluation framework that computes a variety of performance metrics, ranging from true positive and false positive rates to the accuracy of LLM provided explanations.

To demonstrate the utility of this benchmark, we conduct a detailed analysis of major LLMs to answer a range of interesting questions, such as: how well do LLMs perform at investigating security audit logs, and what is the quality of their explanations? Do different tasks or models exhibit performance trends? How do different prompt engineering decisions impact performance on these investigation tasks, and do prompt changes lead to general vs. model-dependent effects? How does the data representation of audit log information impact the performance of LLMs? To what extent do errors overlap across different models, and what are the characteristics of common classes of errors? And, how might security teams and developers most effectively use LLMs for security log investigations?

Overall, we find that popular LLMs achieve mixed performance across the security investigation tasks in our benchmark, with a

tendency towards overly-suspicious verdicts that produce many false positives (§ 5.1). On one hand, several models exhibit strong performance (high true positive rates and low false positive rates) when tasked with identifying instances of data exfiltration. However, across other tasks in our benchmark, LLMs frequently reported a wide spectrum of benign activity as instances of attacks.

Surprisingly, when comparing the performance of the largest LLMs versus smaller versions or open-weight models, our results show that larger models do not always achieve the best performance (§ 5.2). On multiple tasks, smaller models achieved equal or better performance (both higher true positive rates and lower false positive rates) than larger models. Our work also finds that using a more compact preprocessed representation of audit logs leads to strictly better performance for some models across all tasks in our benchmark (§ 5.3). Although this transformed data representation doesn't improve performance for every model, our benchmark and corresponding analysis point to promising directions for practitioners and future work to reduce costs, while maintaining or improving LLM performance for security investigations.

When it comes to comparing and evaluating the performance of models against each other, our results suggest that prompt changes can have model-dependent effects (§ 5.4), similar to how the impact of data representation can vary across models. Although some prompts led to strictly better performance for certain models across all tasks, a given set of prompt changes might cause improvements for some models while reducing the performance of others for the same task and data. Taken together, we caution against using our benchmark, and potentially other benchmarks, as tools to compare models head-to-head without careful exploration and comparisons across the design space. Rather, our experiments and analysis suggest that our benchmark can help developers and security teams study the effects of different design decisions for particular LLMs and systems in practice, and to assess whether any new design patterns enable improvement across a broad set of models.

Finally, our analysis of models' outputs suggests that LLM explanations for true instances of attack activity have fairly high accuracy (§ 6). When they correctly identify attack behavior, LLM explanations tend to capture key attack behaviors reflected in our detailed groundtruth, without including spurious or incorrect details. Nonetheless, our overall evaluation metrics suggest that models tend to flag many instances of benign activity as potential attacks. Based on the LLMs' explanations for these false positives, it appears that models have high sensitivity to any anomalies in system object names and high volume activities (§ 6.2). Future work should explore ways to mitigate high volume of false positives and improve the robustness of LLMs' understanding of attack behavior.

Ultimately, our work provides an initial foundation for studying and improving the capabilities of LLMs for attack investigations. First, we contribute a novel benchmark, AuditBench,[1] including all data, code, documentation, prompts, and model inputs/outputs from our experiments. This benchmark enables both model developers to identify areas for improvement and practitioners to identify appropriate versus cautionary use cases and design patterns to leverage LLMs. Second, we conduct an analysis using our benchmark along these lines, which uncovers opportunities for improving the cost and performance of LLMs for security investigations, as well as insights and advice for making design decisions and cross-model comparisons between LLMs.

[1] https://github.com/aanand300/auditlogsbench

2 Background and Related Work

In this work, we focus on the attack *investigation* process. Whereas detection involves generating alerts, security investigations consist of triaging alerts generated by existing detectors and analyzing different audit logs to understand the full scope of an attacker's activity across an organization's systems [27, 50, 51, 53].

System Audit Logs. As part of an attack investigation, incident response guidelines recommend that security teams should analyze security-related audit logs to uncover key attack activities that require containment or remediation [11, 48, 51]. This log data ranges from sources like network or endpoint intrusion detection systems to logs that operating systems capture via built-in capabilities, such as Linux auditd [36] and Windows ETW or Process Monitoring (procmon) logs [37]. Our work focuses on this latter set of native host logs, specifically Linux auditd and Windows procmon, given their universal applicability and prevalence in academic work [27].

These host logs capture low-level system call activity on a machine, such as read/write/execute operations performed by each process, and network connections initiated or received by each process. Additionally, prior work has developed provenance systems, such as SPADE and CamFlow [20, 43], that can restructure the information in these logs into a graph-based representation with edges that represent a subject, system call action, and an object receiving the action. As discussed in Section 3.4, our work also explores how different representations of log data impact an LLM's performance.

Automating Attack Investigations. Prior work on automating attack investigation has studied using provenance graphs for detection and analysis. These approaches include graph sketching [24] and various machine learning methods, ranging from anomalous node identification [56] to GNNs [10, 44] and masked graph representation learning [28] for improved detection. Despite these advances, using provenance graphs for real-world investigations remains difficult due to the runtime overhead of these systems, as well as the difficulty and time it takes to interpret the attack graph outputs [19]. Unlike potential LLM systems, these prior works require significant human interpretation and do not provide explainable verdicts in text form.

Using LLMs for Log Analysis. Recent work has explored applying LLMs to log parsing, anomalous log event detection, and security-specific applications [1]. For log parsing, systems such as LLM-Parser [35] and LogParser-LLM [65] show that LLMs can convert raw logs into structured formats with minimal rule engineering. In anomaly detection applications, LogPrompt [34] uses instruction-tuned prompts to detect abnormal log patterns and summarize failures without relying on training data. Security-specific applications extend these ideas to threat detection: LogBERT [23], LogFiT [4], and LogLLM [22] adapt transformer models for host-level anomalies, while APT-LLM [6], Audit-LLM [47], and RedChronos [32] apply LLMs to APTs, insider threat, and enterprise-scale log analysis. Collectively, these studies demonstrate growing interest in using

LLMs to automate log analysis and security investigation. Beyond direct log analysis, Kramer et al. [30] and Jones et al. [29] study LLM support for incident response and incident management, showing benefits for analyst assistance but also model-dependent limitations across response stages. Our work advances this space by providing a new benchmark that allows the community to systematically study LLM-driven log analysis across diverse log representations and new security tasks and scenarios.

Security Benchmarks for LLMs. Existing security benchmarks for LLMs focus on three categories: CTF-style challenges [18, 46, 62], vulnerability-focused evaluations (e.g., exploit generation and patching) [31, 57, 61, 66], and cybersecurity knowledge tests assessing factual and procedural understanding [52, 54]. These benchmarks provide valuable insights into how LLMs reason about code and vulnerabilities, but different from our work, their tasks focus on offensive security and isolated software-level challenges, not security investigations. A related line of work, LogEval [13], benchmarks LLMs on general log analysis tasks such as parsing and anomaly detection in AIOps settings. While conceptually close, LogEval focuses on system reliability, fault diagnosis, and similar benign kinds of anomalies, which are fundamentally different than security incidents and attacks. Another work, ExCyTIn-Bench [59], presents a benchmark consisting of logs from Microsoft-specific services such as Azure and MS Sentinel logs. The benchmark constructs granular question-answer pairs for evaluating the performance of LLMs, and provides models with query access to a log database to answer questions about an incident. Our benchmark expands upon this space by broadening the set of logs and security tasks under evaluation. In doing so, we enable novel and complementary findings, as demonstrated by our empirical analysis (e.g., the potential opportunities of pre-processing and transforming log data representations).

Task	Scenario	# Lines (Raw)	# Lines (Edge)	Duration	# TP
Classification	1	1,229,728	5,198	300s	1
	2	238	68	60s	1
	3	8,843	1,493	240s	1
	4	10,980	2,618	600s	1
	5	369,109	2,064	360s	1
Benign	1	6,954	1,857	180s	0
	2	9,394	2,120	300s	0
	3	1,399	509	300s	0
	4	6,616	1,443	300s	0
	5	5,869	1,207	720s	0
Lateral Movement	1	1,782,760	3,189	180s	1
	2	1,126,082	6,569	360s	1
	3	278,413	1,051	240s	1
	4	333,187	1,978	120s	1
	5	193	27	177s	1
Persistence	1	495	140	300s	1
	2	833	188	480s	1
	3	642	304	360s	1
	4	1,538	437	600s	1
	5	771	207	300s	1
Data Exfiltration	1	58	13	120s	1
	2	2,156	947	2,160s	1
	3	15,490	3,801	1,560s	1
	4	2,331	634	300s	1
	5	722,782	3,917	300s	1

Table 1: The Lab dataset portion of our benchmark consists of 25 unique scenarios, with 5 scenarios per security investigation task and 5 scenarios of purely benign activity (§ 3.2).

3 Benchmark Dataset

Our benchmark dataset covers 51 scenarios across four security investigation tasks (§ 3.1). Since our benchmark aims to evaluate performance on *investigation* tasks, our scenarios presume that an analyst and LLM already have a concrete alert or pointer to anchor their analysis around a *targeted set of log lines* [27, 50]. Tables 1 and 2 summarize the size and statistics of our benchmark dataset.

The data consists of operating system logs spanning diverse attack and benign behavior, along with detailed and manually-verified ground truth labels. We generated 25 scenarios on virtual machines (**Lab data**: § 3.2), and curated and labeled an additional 26 scenarios from the public DARPA OpTC dataset [15] (**OpTC data**: § 3.3). For space constraints, we provide a high-level description of our datasets and labeling procedures, with additional details in Appendix A.2. We publicly release our paper's data and code, including ground-truth labels and log-capturing configurations.

3.1 Security Investigation Tasks

Our framework consists of 4 attack investigation tasks. These tasks encompass an alert triaging task and three detailed investigation tasks that security teams perform after discovering an incident. We selected these 4 tasks from recommended actions that consistently appeared in both government incident response frameworks, from NIST [41] and CISA [49], as well as industry guides for threat-hunting and investigation (such as MITRE's threat hunting guide [17]).

(1) Classification (Alert Triaging): given an alert and relevant audit log data, determine whether the alert corresponds to a real attack or a false alarm.
(2) Persistence investigation: identify and explain activity that creates persistence mechanisms in a set of log data (that could allow an attacker to maintain access to the system, as described by MITRE ATT&CK Tactic TA0003 [12]).
(3) Lateral movement investigation: identify and explain all potential lateral movement actions in a set of log data (outbound connections that could provide remote access and control for an attacker; MITRE ATT&CK Tactic TA0008).
(4) Data exfiltration investigation: identify and explain potential events where sensitive user data is exfiltrated from the system in a set of log data (MITRE ATT&CK Tactic TA0010).

Groundtruth Labels. For every scenario, our evaluation framework asks models to output information such as the activity timestamps, key log lines, and explanations for a model's decision. We

Task	Scenario	# Lines (Edge)	Duration	# TP
Classification	1	1,077	10s	1
	2	1,772	4s	1
	3	1,833	21s	1
	4	2,714	118s	1
	5	2,248	93s	1
Benign	1	10,573	60s	0
	2	4,543	60s	0
	3	5,198	60s	0
	4	2,784	60s	0
	5	10,950	60s	0
Velox Benign	1	7,000	20s	0
	2	1,106	20s	0
	3	7,931	20s	0
	4	5,281	20s	0
	5	1,033	20s	0
Lateral Movement	1	355	20s	3
	2	910	20s	9
	3	2,041	20s	2
	4	463	20s	2
	5	953	20s	5
Persistence	1	262	20s	1
	2	2,519	20s	1
	3	3,845	20s	1
	4	1,209	20s	1
Data Exfiltration	1	606	20s	1
	2	2,943	60s	1

Table 2: The OpTC dataset portion of our benchmark consists of 26 scenarios curated and labeled from the DARPA OpTC dataset, including 10 scenarios of benign activity. For each benchmark investigation task, we identified either five scenarios or as many as we could based on the official DARPA groundtruth (§ 3.3).

also developed task-specific groundtruth labels for automatic evaluations that check the following key fields in a model's output.

(1) Classification: a binary value of whether the scenario is malicious or benign.
(2) Persistence: (1) the MITRE ATT&CK name of the persistence technique and (2) the timestamp(s) of where persistence mechanisms were created.
(3) Lateral Movement: (1) the timestamp of an outbound connection with remote execution capabilities on the target device and (2) the host identifier of the target.
(4) Exfiltration: (1) the data (file path and file name) exfiltrated to an external device and (2) timestamp of the data movement.

3.2 Data: Lab data

Although we identified a set of viable scenarios from some public datasets (§ 3.3), many existing datasets such as DARPA TC3 [16] lack sufficient coverage of attack events across our four tasks and/or lack sufficiently precise ground truth labels of the logs. As a result, we created a new audit log dataset of attacks and benign activity on Linux and Windows machines. We constructed 5 distinct attack scenarios for every investigation task to cover diverse real-world attack behavior. Each scenario captures a unique attack technique (or sequence of techniques for some *attack classification* task) based on the MITRE ATT&CK framework [12]. Additionally, we generated five benign scenarios consisting of different, purely benign activities. In total, this Lab dataset consists of audit log data for 25 unique scenarios. As discussed in Section 3.4, scenarios in our Lab data have two different data representations: (1) a **raw** representation (data directly from the native OS log collector), (2) an **edge** representation (after pre-processing).

Scenario Design. To execute the MITRE ATT&CK techniques for a malicious scenario, we studied and used both MITRE ATT&CK's documentation and the Atomic Red Team (ART) framework [9].

The 3 detailed investigation tasks of AuditBench, lateral movement, persistence and exfiltration, each correspond to a high-level tactic in MITRE ATT&CK, and have a list of "techniques" (concrete methods) for executing that tactic. For each investigation task (ATT&CK tactic), we selected 5 distinct techniques that were feasible on Linux or Windows VMs and would generate observable activity in audit logs. For the classification task, we reviewed more techniques across the full set of ATT&CK tactics, and selected ones that clearly indicated compromise of the system so that we could have accurate and automatic binary labels for this task's groundtruth. Four of our classification task scenarios consist of a single ATT&CK technique, whereas the final one consists of a combination of multiple ATT&CK techniques. After identifying the techniques for each scenario, we followed the attack simulation steps from ART [9] for these techniques which occasionally include additional steps (such as deletion of keys after encrypting sensitive files, etc.) in some scenarios to ensure the scenarios were clearly malicious.

Finally, for our five benign scenarios, we simulated different legitimate user operations, such as system administration tasks, normal software installations and programming, file management, and web browsing/network communications. These scenarios cover a mix of GUI-based applications and command-line tools. Appendix A.1 contains additional specific details of our scenarios. The start and end of each scenario corresponds to the start and end of the deliberate benign user activity that was performed.

Data collection. For each scenario, we first documented the list of steps for simulating it (e.g., the applications and commands to execute). Then we simulated these steps using VMs with one of the Windows 10, Windows Server 2022 or Ubuntu 20.04 LTS based on ART's specifications. For log collection, we used ProcMon [37] on Windows machines and Auditd [36] on Linux machines.

After setting up the log collection environment, we simulated attack and benign events based on the steps documented for every scenario. We recorded the timestamps of execution of the steps in

our documentation for building the ground truths(§ 3.1). No other explicit user activity was performed during the execution of the benign or attack steps. We include this documentation as part of our open-sourced data.

3.3 Data: OpTC data

Our benchmark also contains 26 scenarios curated from the DARPA OpTC [15], the largest publicly available system audit log dataset. Compared to other datasets, OpTC contains the most diverse variety of *attack* activity, including lateral movement, persistence, and exfiltration events, and the largest amount of benign activity. The OpTC data creators published a ground truth PDF with a timeline and description of attacks, enabling us to construct attack scenarios and with accurate ground truth labels(§ 3.1). Despite its size, we could identify 4 persistence scenarios and 2 exfiltration scenarios based on a detailed examination of the official ground truth by three researchers. Unlike our Lab data, DARPA OpTC's data comes in only a pre-processed edge representation and does not have the logs in their raw, native OS form.

Attack Scenario Design. To construct our 16 attack scenarios, we initially attempted to directly use the groundtruth to find log lines for the timestamps and hosts with attack activity matching our tasks. Unfortunately, the timestamps of documented malicious activity in the OpTC groundtruth often differed by seconds from when the activity occurred in the log data. As a result we followed a multi-step process with manual verification for constructing attack scenarios from this dataset. First, three researchers reviewed the official groundtruth to identify instances of attack activity that mapped to the four tasks in our benchmark, based on the MITRE ATT&CK tactic definition or where operations clearly indicate malicious behavior that attempts to compromise the system security for the classification task (e.g. credential theft attempts by running known malicious software such as Mimikatz). Next, for each instance that mapped to a potential investigation scenario, the researchers documented as many of the following key identifiers in the groundtruth document for the scenario as we could find (a subset of 8 identifiers: timestamps, file or registry name, PID, process name, IP address, port number, hostname, and principal/username). With this set of identifiers, this set of researchers searched the audit log data for matches close to the reported timestamps to find matching attack "event" log lines. We then constructed each scenario by taking a 20 second window of log line data surrounding the attack event lines (10s before and after).

For some classification and exfiltration tasks, we needed to select longer time durations to ensure that the log lines contained the full activity; for two classification tasks, we had shorter duration scenarios since we found the target attack action in just a single second. Additionally, as shown in Table 2, some investigation scenarios have multiple instances of the attack activity during a scenario's time window (e.g., multiple instances of lateral movement or exfiltration).[2]

[2]We verified all attack activity instances in each scenario by having two researchers create an information flow graph from the scenario's data, and then analyze whether any task-specific attack patterns existed (e.g., for lateral movement, whether an edge had a process make an outbound network connection to an external machine, where the process had remote execution capabilities). Our released dataset also contains documentation describing our detailed groundtruth labeling process.

Benign Scenario Design. Our benchmark contains 10 benign scenarios curated from the OpTC dataset. To construct our first five OpTC benign scenarios, we selected 5 different hosts with no documented attack involvement or activity in the groundtruth, and then selected logs from 60-second windows at a random time on each host. Next, we created an additional five OpTC benign scenarios specifically designed to test an LLM's capabilities to triage false positive detection alerts. For these additional scenarios, we ran a recent state-of-the-art PIDS detector (Velox [7]) on log data from three hosts during a timespan that preceded the first day of any attack activity. Based on this setup, we identified five false positive alerts by selecting five nodes flagged by Velox using its code's preset threshold. We then constructed five benign scenarios by identifying the first occurrence of each of these false positive nodes on the relevant host and taking all log lines in a 20-second window around it (10s before and after).

3.4 Data Representation

As part of our benchmark, we explore two different *representations* of audit log data: a *raw* log and an *edge* representations. The raw representation is the exact log output recorded by native logging tools like Linux's `auditd`. The edge representation is a processed version of this raw data that first converts the raw logs into a provenance or information flow graph. Each line in the edge representation corresponds to an edge in the information flow graph, with a subject node, object node, and action; and data in the edge representation is sorted by timestamp similar to the raw logs. We use a standard provenance tool (SPADE [20]) to generate an edge representation of the raw logs for the Lab portion of our benchmark data.

Exploring both data representations allows for us to have normalized audit log representations across different operating systems (e.g., Linux vs. Windows), and across different datasets (e.g., DARPA OpTC only provides logs in MITRE CAR formats [39, 40], which can easily be transformed into an edge representation but not back to raw logs). Furthermore, the raw Windows procmon logs are simply too verbose and voluminous to be practically used as inputs to LLMs (Table 1), with millions of log lines generated in just a few minutes. In contrast, the edge representation greatly reduces redundant log events and therefore allows us to explore tradeoffs of using compact representations of log data. For our experiments with our Lab data (§ 5), we only run evaluations with the raw logs for scenarios performed on Linux machines; a total of 8 scenarios for our classification task (including benign scenarios), 6 for lateral movement, 10 scenarios for persistence, and 9 for exfiltration.

4 Evaluation Pipeline

Our benchmark takes an LLM (§ 4.1), a scenario's log data (§ 3 and § 4.2), a task-specific prompt (§ 4.2.2), and computes a set of performance metrics by comparing the LLM's output against our groundtruth labels (§ 4.3).

4.1 Models and Parameters

We used our benchmark to evaluate 5 frontier LLMs, including two of the largest state-of-the-art models (Gemini 2.5 Pro and GPT-5) and smaller frontier models (Gemini 2.5 Flash, GPT-5 mini, and

Llama 4 Maverick). To ensure deterministic and minimized variability in responses, we used a temperature of 0 for all experiments when available.[3] For experiments with the GPT-5 models, we set the reasoning effort to the "medium" default.

4.2 Model Inputs

To evaluate a model's performance on an individual scenario, our benchmark runs the model with a common prompt for the scenario's investigation task with the scenario's audit log data. Each prompt specifies a structured JSON output format, which our pipeline automatically compares against our groundtruth labels (§ 3.1).

4.2.1 Log Input. In several of our scenarios, and in many real-world settings, the audit log data might exceed the context window (token limit) for an LLM. As a result, our evaluation pipeline breaks a scenario's audit log data into fixed-sized *chunks* of sequential log lines that (including the task prompt) will fit within context windows of 128,000 tokens (a common minimum window size among many models). When evaluating scenarios on their raw log representation, we used chunks of 1,000 lines; and for evaluations on the edge representation, we used chunks of 400 lines, where these chunk sizes come out to 100,000 - 128,000 tokens based on our data. As described later in Section 5.3, we also evaluated the impact of different chunk sizes for larger-context models.

Our benchmark evaluates a scenario by iteratively evaluating the performance on each chunk (plus the task prompt). It then concatenates the outputs from each chunk into a global set for the scenario and computes final evaluation metrics over this global output set (§ 4.3).

4.2.2 Prompts. For each of the four tasks, our benchmark uses a common prompt to evaluate a model's performance across all the scenarios in a task. Each task's prompt has minor changes for how it describes the audit log data format when evaluating on raw vs. edge representations. In our benchmark and experiments, we studied best practices from several leading LLM platforms [21], [38], [5] and constructed two versions of each task's prompt. In our public code/data, we include both prompt versions and copies of the inputs and outputs for all models across our experiments (§ 5). Future work can easily extend the benchmark or evaluate models using different prompts.

Prompt Version 1. Our first prompt followed a general template with five components: (1) task goals and definitions (e.g., definitions of persistence from the MITRE ATT&CK framework), (2) output format specifications that stated the key fields and what to output, (3) a version of chain-of-thought that required the model to output explanations for the model's reasoning, (4) descriptions of the input data and format, (5) instructions to analyze all of the provided input data and log lines. Four researchers drafted a prompt for each task, then met together to iteratively refine and finalize the prompt (e.g., agreeing on the language to reduce ambiguities and ensure output requirements matched our groundtruth formatting). Appendix B provides the general template and an example of the prompt used for the exfiltration task.

[3]GPT-5 does not provide temperature values other than the default value of 1.

Prompt Version 2. We also designed a second version of each task's prompt, as a way to explore the impact of prompt engineering and to simulate a practical setting where an analyst refined their prompts based on their initial usage. Specifically, two researchers (including one not involved in the first prompt version creation) reviewed the latest prompt engineering best practices [5, 21, 38] and edited the Version 1 prompts in four ways. First, these researchers interactively refined each task's prompt by looking at a set of false positive errors of two scenarios with higher FPs in each task from our Lab dataset. Second, we increased the specificity of each of the three detailed investigation tasks by including more precise descriptions from MITRE ATT&CK and some in-context examples about the target behavior; we did not find any common limitations or necessary clarifications for the classification task. Third, we added additional instructions to elicit and produce multi-step reasoning and deliberation before outputting a final conclusion [33, 42]. Finally, we added a model persona (computer security expert) and modified the prompt to follow a markdown formatting [25]. Appendix B.3 shows the general template for the Version 2 prompts and our exfiltration task's prompt for this version.

4.3 Evaluation Metrics

Using our groundtruth labels, our benchmark quantifies the performance of LLMs on our benchmark tasks by computing the **True Positive Rate (TPR)**, **False Positive Rate (FPR)**, and **F1 score** [58]. For each task, and a given data representation, the TPR equals the fraction of total attack instances correctly identified by the LLM across all scenarios in the task. In most attack scenarios, there is only one attack instance in our groundtruth for each scenario; but some scenarios have multiple attack instances in a scenario (e.g., the Lateral Movement scenarios in our OpTC derived data). Our benchmark's automatic evaluator parses a model's output and compares the key identifiers for each task against the groundtruth (§ 3.1) and counts how many unique groundtruth entities the model's output contains (divided by the total number of unique entities across a task).[4] Because our classification task consists of a binary label per scenario, we score it asymmetrically: a model receives one true positive per attack scenario in which it flags *any* chunk as malicious, while each chunk in a benign scenario that it flags as malicious counts as a separate false positive.

4.4 Limitations

Our benchmark's dataset does not cover all possible attacks a security team might investigate, and it does not test attacks that aim to deliberately evade LLMs. For our classification task, our benchmark also does not evaluate attacks where the necessary log information spans very long durations, due to the prohibitive cost for both researchers and practitioners to feasibly use LLMs for these cases. Additionally, our benign scenarios may not capture the full space of all benign activity and false alarms in practice. Nonetheless, our benchmark makes an important contribution by presenting a set of well-defined tasks for this problem and a novel dataset with detailed groundtruth. Future work can continue to expand upon

[4]Our evaluation framework uses both a standard JSON parser, as well as a custom parser designed to correctly handle and score LLM outputs that have errors in their JSON format. We manually verified that our framework's parsing correctly and automatically scores LLM responses across all scenarios and models we tested.

Table 3: Summary of LLM performances across our benchmark's Lab data (§ 3.2). Each row shows the performance metric value for the LLMs across a task and data representation (Column 2: "Rep"). We highlight and bold the best value for each row (highest F1 score, highest TPR, or lowest FPR). For the classification task, the false positive denominators reflect the number of log chunks; while for the other three tasks, the denominator reflects the total number of log lines (§ 4.3).

Task	Rep	Metric	Gpt-5-mini, Prompt v2	Gemini-2.5-flash, Prompt v2	Llama 4 Maverick, Prompt v2	Gpt-5, Prompt v2	Gemini-2.5-pro, Prompt v2
classification	edge	F1	0.57	0.36	**1.00**	0.77	0.36
classification	edge	TPR	2/5 (40.00%)	**5/5 (100.00%)**	**5/5 (100.00%)**	**5/5 (100.00%)**	**5/5 (100.00%)**
classification	edge	FPR	**0/21 (0.00%)**	18/21 (85.71%)	**0/21 (0.00%)**	3/21 (14.29%)	18/21 (85.71%)
lateral movement	edge	F1	0.57	0.48	**0.62**	0.33	0.45
lateral movement	edge	TPR	2/5 (40.00%)	**5/5 (100.00%)**	4/5 (80.00%)	1/5 (20.00%)	**5/5 (100.00%)**
lateral movement	edge	FPR	**0/19950 (0.00%)**	11/19950 (0.06%)	4/19950 (0.02%)	**0/19950 (0.00%)**	12/19950 (0.06%)
persistence	edge	F1	**0.80**	0.67	0.57	0.57	0.47
persistence	edge	TPR	**4/5 (80.00%)**	**4/5 (80.00%)**	2/5 (40.00%)	2/5 (40.00%)	**4/5 (80.00%)**
persistence	edge	FPR	1/8412 (0.01%)	3/8412 (0.04%)	**0/8412 (0.00%)**	**0/8412 (0.00%)**	8/8412 (0.10%)
exfiltration	edge	F1	0.50	0.56	0.29	**0.77**	0.40
exfiltration	edge	TPR	3/5 (60.00%)	**5/5 (100.00%)**	1/5 (20.00%)	**5/5 (100.00%)**	**5/5 (100.00%)**
exfiltration	edge	FPR	4/16448 (0.02%)	8/16448 (0.05%)	**1/16448 (0.01%)**	3/16448 (0.02%)	15/16448 (0.09%)
classification	raw	F1	**1.00**	0.75	0.80	0.80	0.46
classification	raw	TPR	**3/3 (100.00%)**	**3/3 (100.00%)**	2/3 (66.67%)	2/3 (66.67%)	**3/3 (100.00%)**
classification	raw	FPR	**0/32 (0.00%)**	2/32 (6.25%)	**0/32 (0.00%)**	**0/32 (0.00%)**	7/32 (21.88%)
lateral movement	raw	F1	0.00	0.00	0.00	0.00	**1.00**
lateral movement	raw	TPR	0/1 (0.00%)	0/1 (0.00%)	0/1 (0.00%)	0/1 (0.00%)	**1/1 (100.00%)**
lateral movement	raw	FPR	1/30425 (0.00%)	4/30425 (0.01%)	**0/30425 (0.00%)**	**0/30425 (0.00%)**	**0/30425 (0.00%)**
persistence	raw	F1	0.57	0.60	0.33	0.50	**1.00**
persistence	raw	TPR	2/5 (40.00%)	3/5 (60.00%)	1/5 (20.00%)	2/5 (40.00%)	**5/5 (100.00%)**
persistence	raw	FPR	**0/34511 (0.00%)**	2/34511 (0.01%)	**0/34511 (0.00%)**	1/34511 (0.00%)	**0/34511 (0.00%)**
exfiltration	raw	F1	0.62	0.62	0.29	**0.67**	0.57
exfiltration	raw	TPR	**4/4 (100.00%)**	**4/4 (100.00%)**	1/4 (25.00%)	3/4 (75.00%)	**4/4 (100.00%)**
exfiltration	raw	FPR	5/50267 (0.01%)	5/50267 (0.01%)	**2/50267 (0.00%)**	**2/50267 (0.00%)**	6/50267 (0.01%)

our dataset, which spans both newly generated data and carefully curated public data sources, multiple benign scenarios specifically designed to simulate false positives from a detector (§ 3.3), and dozens of different attack behaviors.

Our experiments (§ 5) do not test every possible LLM, prompt, or application architecture, since it is impossible to exhaustively cover the space. Instead, we selected a reasonably representative set of the largest state-of-the-art models (at the time of our experiments) and smaller frontier models, and devised different prompts following prompt engineering best practices. Additionally due to costs, our experiments evaluate the models using a temperature of 0 (when available) to achieve deterministic results. Our experiments aim to provide a first look and new insights into the different subtleties and design considerations that developers and researchers should take into account when using/designing LLMs for security investigations. We acknowledge that newer models have and will continue to be released. By open sourcing all our data and code (including model inputs/outputs), we provide a necessary foundation for future work to study this space and analyze the performance of new models.

5 Results and Analysis

This section presents the results and analysis of evaluating five popular LLMs on our benchmark dataset. First, we find that no model achieves strictly better performance across all tasks (§ 5.1), with models frequently exhibiting overly-suspicious and false-positive prone verdicts across many tasks. Second, contrary to scaling expectations, smaller LLMs showed competitive and sometimes better performance than larger models (§ 5.2). Third, using a more efficient edge data representation strictly improves performance for some models; however, we do see model-specific and task-specific effects (§ 5.3). Finally, changes to the prompts for a task can also have model-dependent effects (§ 5.4). Thus, comparing performance between models requires careful exploration of different designs and configurations, rather than a one-size-fits-all approach.

5.1 Benchmark Difficulty and Overall Performance Results

Tables 3 and 4 show the performance of LLMs on the Lab and OpTC portions of our dataset respectively. Overall, no model dominates across all tasks, where different models achieve the best F1, TPR, and FPR values depending on the task and data representation.

In terms of difficulty, our benchmark's data and tasks seem to present meaningful challenges for recent frontier models. Only a

Table 4: Summary of LLM performances on our benchmark's OpTC data (§ 3.3). For the classification task, the false positive denominators reflect the number of log chunks; while for the other three tasks, the denominator reflects the total log lines (§ 4.3).

Task	Rep	Metric	Gpt-5-mini, Prompt v2	Gemini-2.5-flash, Prompt v2	Llama 4 Maverick, Prompt v2	Gpt-5, Prompt v2	Gemini-2.5-pro, Prompt v2
classification	edge	F1	0.45	0.08	0.29	0.45	0.08
classification	edge	TPR	5/5 (100.00%)	5/5 (100.00%)	5/5 (100.00%)	5/5 (100.00%)	5/5 (100.00%)
classification	edge	FPR	12/145 (8.28%)	115/145 (79.31%)	24/145 (16.55%)	12/145 (8.28%)	116/145 (80.00%)
lateral movement	edge	F1	0.18	0.25	0.05	0.08	0.20
lateral movement	edge	TPR	3/19 (15.79%)	7/19 (36.84%)	1/19 (5.26%)	1/19 (5.26%)	6/19 (31.58%)
lateral movement	edge	FPR	12/61121 (0.02%)	30/61121 (0.05%)	15/61121 (0.02%)	3/61121 (0.00%)	33/61121 (0.05%)
persistence	edge	F1	0.43	0.21	0.17	0.60	0.20
persistence	edge	TPR	3/4 (75.00%)	3/4 (75.00%)	2/4 (50.00%)	3/4 (75.00%)	4/4 (100.00%)
persistence	edge	FPR	7/64234 (0.01%)	22/64234 (0.03%)	18/64234 (0.03%)	3/64234 (0.00%)	34/64234 (0.05%)
exfiltration	edge	F1	1.00	0.29	0.50	1.00	0.15
exfiltration	edge	TPR	2/2 (100.00%)	2/2 (100.00%)	2/2 (100.00%)	2/2 (100.00%)	2/2 (100.00%)
exfiltration	edge	FPR	0/59948 (0.00%)	10/59948 (0.02%)	4/59948 (0.01%)	0/59948 (0.00%)	22/59948 (0.04%)

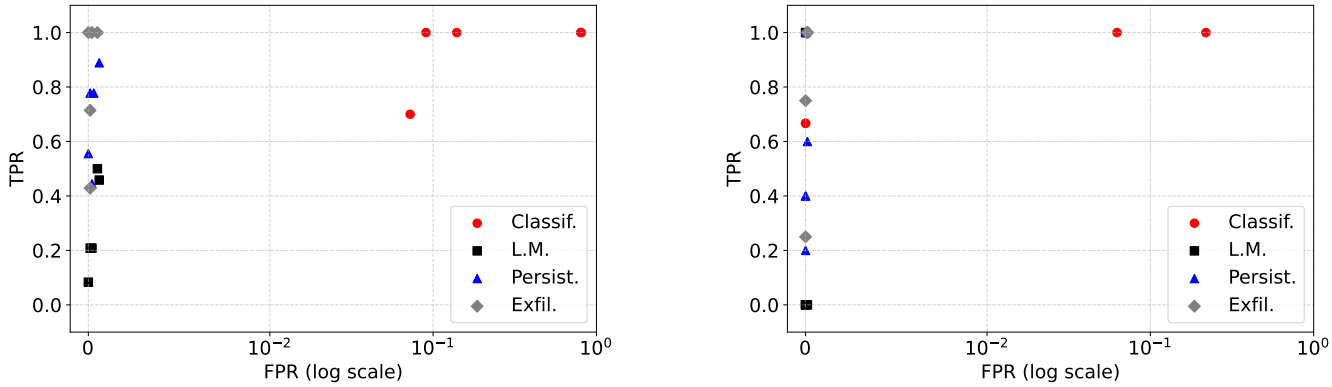

Figure 1: Performance of the 5 different LLMs across the benchmark's tasks (§ 5.1); each point represents one LLM on a task, where some points overlap due to equal performance. Fig (a) shows the performance on tasks given the edge representation, while Fig (b) shows the raw representation performance. The top left corner represents optimal performance, with lower false positive rates (x-axis) and higher true positive rates (y-axis).

few tasks have models with perfect performance (e.g., classification on the Lab data and exfiltration on the OpTC data); and even for these tasks, most models produce errors. The OpTC scenarios generally lead to worse performance on the classification, lateral movement, and persistence tasks, while the exfiltration scenarios in our Lab data seem more difficult compared to the task's OpTC scenarios, with max F1-scores of 0.67 vs. 1.0 respectively.

Additionally, as seen in Figure 1, model performance varies notably across tasks, although the relative difficulty of each task remains fairly consistent across LLMs. LLMs performed the best on the exfiltration task, followed by the persistence task, where the top left of Figure 1, with higher TPR and lower FPR, represent optimal performance. In contrast, models tend to perform worst on average for the lateral movement task, which yields the lowest median F1-score across the five LLMs. Finally, most notably in the OpTC portion of our dataset, most models tend to produce fairly high false positive rates on both the lateral movement and classification tasks, with dozens of false positives across several thousands of log lines.

5.2 Impact of Model Size

Surprisingly, our experiments suggest that the larger LLMs do not always achieve the best results when compared to much smaller models. At the time of our experiments, GPT-5 and Gemini 2.5 Pro were two of the largest frontier LLMs, whereas GPT-5 mini, Gemini 2.5 Flash, and Llama 4 Maverick represent high-performing but smaller models. But as seen in Tables 5 and 6, on a specific task, the best-performing smaller model can achieve higher or equivalent performance than the best performing larger model when analyzing the edge representation of the data. This competitiveness holds across both the Lab and OpTC portions of our benchmark data. For practitioners seeking to use LLMs to aid security investigations, this finding suggests that they might not need to use larger and more expensive models for optimal performance.

Table 5: A simplified version of Tables 3 and 4 that compares the highest F1 score on our benchmark's Lab data across smaller models (GPT-5 mini, Gemini 2.5 Flash, Llama 4 Maverick) vs. larger models (§ 5.2). For the lateral movement task using raw logs, we have only one testable attack scenario, with one true positive (TP); so the max F1 score of 0 reflects all small models missing the TP.

Task	Rep	Metric	Max F1 (smaller models)	Max F1 (larger models)
classification	edge	F1	**1.00**	0.77
lateral movement	edge	F1	**0.62**	0.45
persistence	edge	F1	**0.80**	0.57
exfiltration	edge	F1	0.56	**0.77**
classification	raw	F1	**1.00**	0.80
lateral movement	raw	F1	0.00	**1.00**
persistence	raw	F1	0.60	**1.00**
exfiltration	raw	F1	0.62	**0.67**

Table 6: Comparisons of the best smaller model vs. larger model performance on our benchmark's OpTC data. Overall smaller models often perform on-par with the largest frontier models (§ 5.2).

Task	Rep	Metric	Max F1 (smaller models)	Max F1 (larger models)
classification	edge	F1	**0.45**	**0.45**
lateral movement	edge	F1	**0.25**	0.21
persistence	edge	F1	0.43	**0.60**
exfiltration	edge	F1	**1.00**	**1.00**

Table 7: The effect of chunk sizes on Gemini 2.5 Flash and Gemini 2.5 Pro for the edge representation using Prompt v2 (§ 5.3). Across both models, our default choice of 400-line chunks performs commensurate or better than alternative sizes.

Task	Model	Metric	200 lines	400 lines	800 lines	1600 lines
Classification	Gem 2.5 Flash	TPR FPR	5/5 (100.00) 26/39 (66.67)	5/5 (100.00) 18/21 (85.71)	5/5 (100.00) 10/11 (90.91)	5/5 (100.00) 7/7 (100.00)
L.M.	Gem 2.5 Flash	TPR FPR	4/5 (80.00) 25/19950 (0.13)	5/5 (100.00) 11/19950 (0.06)	5/5 (100.00) 8/19950 (0.04)	5/5 (100.00) 6/19950 (0.03)
Persistence	Gem 2.5 Flash	TPR FPR	5/5 (100.00) 3/8412 (0.04)	4/5 (80.00) 4/8412 (0.05)	3/5 (60.00) 5/8412 (0.06)	3/5 (60.00) 5/8412 (0.06)
Exfiltration	Gem 2.5 Flash	TPR FPR	2/5 (40.00) 10/16448 (0.06)	5/5 (100.00) 8/16448 (0.05)	3/5 (60.00) 10/16448 (0.06)	3/5 (60.00) 8/16448 (0.05)
Classification	Gem 2.5 Pro	TPR FPR	5/5 (100.00) 29/39 (74.36)	5/5 (100.00) 18/21 (85.71)	5/5 (100.00) 11/11 (100.00)	5/5 (100.00) 7/7 (100.00)
L.M.	Gem 2.5 Pro	TPR FPR	4/5 (80.00) 17/19950 (0.09)	5/5 (100.00) 12/19950 (0.06)	5/5 (100.00) 6/19950 (0.03)	5/5 (100.00) 2/19950 (0.01)
Persistence	Gem 2.5 Pro	TPR FPR	5/5 (100.00) 10/8412 (0.12)	4/5 (80.00) 8/8412 (0.10)	5/5 (100.00) 8/8412 (0.10)	5/5 (100.00) 5/8412 (0.06)
Exfiltration	Gem 2.5 Pro	TPR FPR	2/5 (40.00) 19/16448 (0.12)	5/5 (100.00) 15/16448 (0.09)	2/5 (40.00) 14/16448 (0.09)	2/5 (40.00) 4/16448 (0.02)

5.3 Impact of Data Representations

Table 8 compares the performance of using different data representations, edge vs. raw (§ 3.4), across the different models. For each task, we only compare the scenarios that had feasible raw representations; e.g., the OpTC dataset does not provide raw logs and some of the Lab scenarios generated on Windows machines have prohibitively large volumes of raw log lines (§ 3).

Overall, the best-performing data representation can depend on the specific model and task. For example, Gemini 2.5 Pro performs better when given the raw log data for all tasks. However, for many other models, the more efficient edge representation leads to equal or better performance on many tasks; and for Llama 4, the edge representation strictly improves performance. This novel finding suggests that future work and practitioners should explore how different data pre-processing and transformation approaches can improve not only costs, but also performance.

Chunk Size Sensitivity. Due to the max context windows of some models, our benchmark partitions audit logs into chunks of approximately 100,000 tokens for evaluation (§ 4.2). The Gemini 2.5 Flash and Pro models support context windows of up to 1M tokens. To evaluate the impact of chunk size (log lines per chunk), we conducted a sensitivity analysis using both models across our benchmark. As summarized in Table 7 and Table 11, performance remains fairly stable across different chunk sizes. Occasionally the smallest chunk size omits necessary context; but our default, middle chunk size avoids this problem (e.g., 400 lines for edge representations).

5.4 Effects of Prompt Variations

So far, we used the Prompt v2 variation of our prompts (§ 4.2.2) when presenting results. In this prompt version, the final verdict for each event can be one of three labels: "low", "medium", or "high" suspicion. For our experimental results, we only considered entities with "high" suspicion verdicts as entities the model flags as an attack. Events where the model assigns a "low" or "medium" verdict are treated as events the model considers benign (either true negatives or false negatives), along with anything the model doesn't report. In this section, we explore the impact of these two choices: the change in performance if we use Version 1 of our prompt and the impact of adopting a more aggressive verdict interpretation (e.g., treating "medium suspicious" events as activity flagged by the LLM).

Prompt Versions. Table 9 compares the performance of using Prompt v1 versus Prompt v2 across all scenarios in our benchmark for four LLMs. (In these experiments, we omit GPT-5 due to costs, but our analysis still includes GPT-5 mini and a different large LLM: Gemini 2.5 Pro). In most cases, Prompt v2 leads to better results. For the persistence and exfiltration tasks, this second prompt version led to strictly better F1-scores for almost all models and both data representations. Additionally, for two smaller LLMs (GPT-5 mini and Llama 4 Maverick), the second prompt leads to optimal performance on all tasks when given the data's edge representation. In all these cases, the F1-score increases due to reductions in the false positive rates, with often no change to the true positive rates.

We attribute most of Prompt v2's gains to its added task-specific specificity (e.g. specifying known attacker behavior like cloud uploads, HTTPS exfiltration, pre-exfiltration compression for exfiltration; and emphasizing suspicious *outgoing* connections rather than DNS/CDN traffic for lateral movement) and explicitly scoping out common false-positive sources (e.g., system-generated and temporary files that we considered not-sensitive in our specific investigation setup for exfiltration). By providing these specific details, we aim to incorporate domain knowledge into the model's

Table 8: Comparisons of LLM performance when given edge vs. raw data representations (§ 5.3). Green cells show the highest model performance per task (row) for each model. For this comparison, we only analyze scenarios which have both raw and edge representations.

Task	Metric	GPT-5 mini edge	GPT-5 mini raw	Gemini 2.5 Flash edge	Gemini 2.5 Flash raw	Llama 4 Maverick edge	Llama 4 Maverick raw	GPT-5 edge	GPT-5 raw	Gemini 2.5 Pro edge	Gemini 2.5 Pro raw	Comparable scenarios
classification	F1	0	**1.00**	0.25	**0.75**	**1.00**	0.80	0.67	**0.80**	0.25	**0.46**	8
lateral movement	F1	**0**	**0**	**0.40**	0	**1.00**	0	**0**	**0**	**1.00**	**1.00**	6
persistence	F1	**0.80**	0.57	**0.67**	0.60	**0.57**	0.33	**0.57**	0.50	0.47	**1.00**	10
exfiltration	F1	0.55	**0.62**	0.50	**0.62**	**0.33**	0.29	**0.73**	0.67	0.35	**0.57**	9

Table 9: Comparisons of the performance for each LLM between our two prompt versions (§ 4.2.2). The edge representation rows in this table show the performance over all scenarios across the Lab and OpTC datasets. Highlighted green cells indicate the best (or equal) performance between the prompts for each model. We replaced undefined F1 scores (caused by a model detecting 0 true positives) with a value of 0; in particular, for raw log representations, the lateral movement task had only 1 attack scenario that could be feasibly tested.

Task	Rep	Metric	Gpt-5-mini, Prompt v1	Gpt-5-mini, Prompt v2	Gemini-2.5-flash, Prompt v1	Gemini-2.5-flash, Prompt v2	Llama 4 Maverick, Prompt v1	Llama 4 Maverick, Prompt v2	Gemini-2.5-pro, Prompt v1	Gemini-2.5-pro, Prompt v2
classification	edge	F1	**0.48**	**0.48**	**0.22**	0.13	0.17	**0.45**	**0.14**	0.13
lateral movement	edge	F1	0.20	**0.24**	0.21	**0.31**	0.06	**0.21**	**0.29**	0.28
persistence	edge	F1	0.12	**0.58**	0.11	**0.34**	0.08	**0.26**	0.09	**0.27**
exfiltration	edge	F1	0.12	**0.62**	0.13	**0.44**	0.04	**0.40**	0.14	**0.27**
classification	raw	F1	**1.00**	**1.00**	**1.00**	0.75	**1.00**	0.80	**0.55**	0.46
lateral movement	raw	F1	**0.67**	0.00	**0.17**	0.00	**0.00**	**0.00**	**1.00**	**1.00**
persistence	raw	F1	0.56	**0.57**	0.36	**0.60**	**0.44**	0.33	0.24	**1.00**
exfiltration	raw	F1	0.29	**0.62**	0.35	**0.62**	0.06	**0.29**	0.28	**0.57**

computation, which might not be well-defined or internalized during an LLM's existing training or RLHF instruction tuning.

However, we note that prompt changes do not always have uniform effects across all models. For example, on the classification task when using the edge representation, Prompt v2 improves Llama 4's performance significantly, but leads to worse performance on Gemini 2.5 Flash. This mixed effect suggests that prompts can have noticeable model-dependent impacts, and that a one-size-fits-all-approach may not accurately compare models head-to-head.

Aggressive Verdict Interpretations. Our results so far use a conservative interpretation of the model's verdict (treating only "high suspicious" as attack outputs). However, analysts could go with a more aggressive interpretation that also treats "medium" suspicion as attacks, achieving higher true positives at the cost of more false positives. We evaluated the impact of switching to this more aggressive prompt, and found this change leads to strictly worse performance for most models across almost all tasks and data representations; true positive rates often remained the same, while the volume of false positives increased. However, we did observe some rare model-specific effects with this design choice; unlike other models, GPT-5 had almost uniformly better performance when analyzing the raw audit log data with this more aggressive prompt version. Table 12 in the Appendix provides more granular details.

6 Analysis

In this section, we analyze the explanations in the models' outputs to better understand both the quality of LLM-generated explanations for true positives (Section 6.1) and the thematic characteristics in the errors that the LLMs made on investigation tasks (Section 6.2).

6.1 Analysis of LLM-generated explanations

As described in Section 4.2.2, our prompt instructs each LLM to generate a `"deliberation"` field before producing its final verdict. If accurate, these explanations can help analysts interpret why a model flagged an activity as highly suspicious, and facilitate faster and more accurate investigations. In this subsection, we evaluate the correctness of LLM-generated explanations across true-positive activity they identify across all our models and tasks.

Measuring Explanation Quality: To compute explanation quality, we treat the problem as a Natural Language Inference (NLI) task [14], also known as textual entailment, and apply an LLM-as-a-judge for assessing textual entailment. Here the *hypothesis* is the LLM-generated explanation, and the *premise* is the ground truth attack description. An explanation (hypothesis) is entailed if the text of the ground truth (premise) semantically supports its claims (about existence of specific attack behaviors and artifacts).

To construct each scenario's entailment "premise", our benchmark uses Gemini 2.5 Pro to produce a single paragraph that combined the ATT&CK technique description with the detailed description of the list of attack steps we performed and recorded during our scenario generation (which includes details like timestamps, execution commands, etc.). To facilitate fine-grained explanation assessment, we compute entailment scores for any chunk with true

Table 10: Entailment scores of model-generated explanations across the benchmark tasks, on a scale of 1 – 5, where a score of 5 represents the highest explanation accuracy relative to the groundtruth (§ 6.1).

	Classification	Lat. Movement	Persistence	Exfiltration
gemini-2.5-flash	3.34	4.80	4.12	3.96
gemini-2.5-pro	3.36	4.40	4.00	4.00
gpt-5	4.00	5.00	4.50	4.52
gpt-5-mini	4.00	4.00	4.50	3.67
llama-4-maverick	4.27	5.00	5.00	5.00
Avg. LLM score	3.79	4.64	4.42	4.23

positives by treating the chunk's LLM-generated explanation as the entailment hypothesis. Our benchmark computes quality (entailment) scores on a five-point Likert scale [64], where 1 indicates no entailment (poor quality) and 5 indicates complete entailment (high quality).

We then used Gemini 2.5 Flash as a LLM judge with a cybersecurity expert persona to score each premise-hypothesis pair (the quality of each chunk's explanation across true positives for each model and all scenarios). The judge prompt instructs the LLM to increase the score when the LLM explanations contain legitimate attack relevant behaviors and low-level artifacts (e.g., malicious file/process names). In contrast, the prompt instructs the LLM-judge to penalize the score when the explanation contradicts the attack or introduces irrelevant details about benign activity (e.g., PIDs or file names that do not relate to any attack activity).

Explanation Quality Results: As seen in Table 10, LLMs have fairly good explanation quality across the benchmark for true positives they correctly identify. Explanations for the classification task have a lower average entailment score (3.79/5) than those for lateral movement (4.64/5), persistence (4.42/5), and exfiltration (4.23/5).

To validate the accuracy of these LLM-judged quality scores, two authors manually rated a random sample of explanations (30 premise-hypothesis pairs) using the same 1-5 scale. Human raters achieved a weighted Cohen's κ of 0.409, while the judge reached κ of 0.575 against the human consensus; both comparisons had 93.3% adjacent agreement (within ±1 point). On average, the LLM-judge approach in our framework errs towards giving lower quality ratings (-0.2 points) than the human raters. Overall, these numbers suggest our framework's automated approach to computing explanation quality is reasonably reliable.

6.2 Model Error Analysis

In this section, we examine the extent to which errors overlap across models and the characteristics of common errors. We focus specifically on false positive errors, since they constitute the larger class of errors (Tables 3 and 4), and represent one of the biggest challenges and bottlenecks for security teams in practice [3]. Additionally, false positive errors produce concrete entities that we can compare across models, and include specific reasoning. In contrast, false negatives by their nature often have an absence of output or specific suspicious entities (since models consider all activity to be benign), which makes it difficult to study false negative errors.

Figure 2 shows the number of distinct false positives (FPs) generated by exactly 1, 2, 3, 4, or all 5 models, across all scenarios for each task in our benchmark when using the edge representation of audit logs. For the most part, the same error rarely gets flagged by a majority of models: a total of 350 unique FPs across all tasks only occur for 1 or 2 models. When processing the edge representation, only 19 FPs get produced by four or more models (14 false positives from the classification task, 1 from the lateral movement task, and 4 from the persistence task). For scenarios evaluated on the raw log representation (not shown in the figure), we observed even smaller error overlap (due in part to the smaller set of scenarios with available raw logs). Among the 37 unique false positives across all tasks, none were flagged by three or more models, and only 1 false positive was produced by two models.

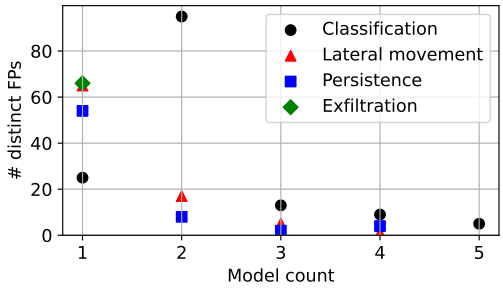

Figure 2: The number of distinct false positives (y-axis) that exactly N models generate (x-axis), for each task across our benchmark when using the data's edge representation. These numbers include all false positives, across both the Lab and OpTC portions of our benchmark.

Additionally, we saw minimal overlap between the errors produced by models when given two different data representations of the same scenario. Among 75 total FPs from all models on all scenarios with both raw and edge representations, only one FP (in a persistence scenario) was common across the errors generated from processing the edge representation and when processing the raw logs. This lack of overlap suggests distinct differences in how LLMs interpret security logs, and provides potential opportunities to defenders. For example, practitioners may be able to use ensembles of different models to achieve lower false positive rates.

6.3 Characteristics of Common Errors

To understand the reasons for common errors, we manually investigated the 19 "common" false positives generated by 4 or more LLMs; all of these errors come from the OpTC data in our benchmark. Our prompts instruct the models to output reasoning traces for their decisions (e.g., in output fields that ask an LLM to specify the "key log lines", "evidence", and a "deliberation" before the model outputs its verdicts). We carefully examined these reasoning-related fields (and verified their factual consistency with the input logs) to better understand the characteristics of common errors.

When analyzing our LLMs' reasoning outputs for common false positives, we observed that the text consistently relied on two characteristics about the logging data:

- The names of system objects in the logs, such as the names of processes, scripts, and/or tasks executed.

- The sequence of events in the logs. In some cases, the reasoning traces also highlight the volume of an activity in a short time window as a suspicious sign.

Concretely, 18 / 19 of the common false positives have a heavy reliance on system object names in the reasoning traces of many models. In one common classification task false positive, GPT-5 mini's deliberation highlights "non-standard" names in the logs, such as `C:\\ncr\\outlook\\invis.vbs` (name of a script), `login_keys_64bit.bat`, `popup_keys_64bit.bat` (names of batch files that were executed), as suspicious signs. Additionally, the model states that these "names strongly suggesting credential or key-capture behavior". As another example, for a common false positive among the persistence scenarios, Gemini 2.5 Pro's deliberation states: "The most damning evidence is the execution of a scheduled task explicitly named `\\clear_logs`". For this same FP instance, GPT-5 stated the following in its deliberation: "a scheduled task named `\\\\clear_logs` runs a SYSTEM-level batch file at `C:\\ncr\\DeleteArchiveSecurity.bat`, whose name implies deletion of security log archives". This output clearly shows GPT-5's reliance on the name of the batch file.

Similarly, across 17 of the 19 common false positives, the sequence of events appears frequently in model's explanation-related outputs. In these cases, the deliberation field frequently mentioned terms such as "sequence", "chain", "execution flow", and referenced parent nodes and temporal relationships between entities. For instance, the deliberation sentence of Gemini 2.5 Flash on one of the common classification task false positives states: "The fact that this script is executed by 'cmd.exe' processes spawned by 'svchost.exe', 'csrss.exe', and 'schtasks.exe' with SYSTEM privileges is highly alarming." In another false positive instance, Gemini 2.5 Flash reports that "The core of the suspicious activity lies in the *execution chain*: 'Explorer.EXE' → 'WScript.exe' → 'invis.vbs' and 'login_keys_64bit.bat'". It then states that this sequence is a key characteristic for attacks such as "credential/keylogging or stealthy post-compromise scripting".

When manually analyzing these common false positives, we identified four false positives corresponding to security relevant activity that could be difficult to unambiguously label without external context. The activities included disabling system integrity checks, modifying Outlook anti-spam settings, and a powershell-launched outbound TCP connection. But they all occur in scenarios created from the DARPA OpTC dataset, and from portions of that dataset explicitly labeled as benign by the official OpTC distributors. We respect the original DARPA labels but we recognize that these cases might represent situations where different security teams may want to interpret and act on the activity differently. Such ambiguity will inevitably occur in real-world logs across organizations. The publicly released data and code for our benchmark include the groundtruth labels, documentation describing our labeling process, and the functions for computing evaluation metrics. As a result, users and future work can modify this decision depending on their situation and goals. Additionally, we carefully constructed the 25 Lab scenarios in our data to avoid any potentially ambiguous activity.

7 Discussion and Conclusion

In this work, we developed a novel benchmark for evaluating the use of LLMs in security investigations. Our benchmark covers four common security investigation tasks, consisting of newly generated data and curated data from the most complex, publicly available security audit log dataset. We believe our benchmark serves as a useful measurement tool for researchers and practitioners to evaluate and improve LLMs' capabilities at security investigations. As an example, we distill several interesting findings based on using our benchmark to analyze the performance of five popular LLMs.

First, our evaluation suggests that models appear to have an overly-suspicious interpretation of audit log data, incurring high numbers of false positives for some tasks (§ 5.1). Although some models can achieve both high true positives and low false positives when identifying exfiltration behavior, the LLMs we test frequently report many benign activities as potential attacks across other tasks. Based on the explanation outputs from LLMs, these false positives often appear to over-index on potentially unusual names in files, processes, and other objects, as well as high-volume activities (§ 6.3). Future work should explore ways to improve the robustness of models' understanding of attack behavior and reduce false positives. Given the small overlap in errors, one potential direction for mitigating false positives may be to explore using an ensemble of models to produce alerts (§ 6.2).

Second, our results point to several directions for improving both the cost and performance of using LLMs for security investigations. For example, we found that for many tasks, smaller models can perform just as well as larger models (§ 5.2). Additionally, our work finds that for certain models, including the smaller, open-weight Llama 4 model, using a transformed and more efficient representation of log data leads to strictly better performance (§ 5.3). These findings indicate that data representation engineering, and combining insights from existing work on provenance graphs and information flow with LLMs could be promising avenues for future work, as ways to improve the cost, efficiency, and performance of LLMs on security investigation tasks.

Third, our experiments provide cautionary advice for comparing or choosing between specific LLMs to use. In particular, our evaluations show that various design decisions can lead to model-specific performance changes. For example, while prompt changes that improve the performance of one model tend to improve the performance of all models (§ 5.4), we observed some deviations from this trend across our benchmark. Similarly, the transformed edge representation of our data led to strict improvements for some models, but worse performance for others. Thus, we believe that practitioners and researchers need to carefully explore a range of different configurations and designs, and weigh the optimal choice for each model when making cross-model comparisons in practice. Our datasets and benchmark provide the means to perform this assessment and decision making.

Ultimately, this paper contributes both an extensible benchmark and novel insights for security teams, model developers, and researchers exploring how to use and improve LLMs for security investigations. We publicly release all our code, data, documentation, prompts, and model input/outputs as part of our benchmark contribution. We hope that future work continues to use and extend

our benchmark as a means to study and improve the use of LLMs for investigating security audit logs.

References

[1] Siraaj Akhtar, Saad Khan, and Simon Parkinson. 2025. LLM-based event log analysis techniques: A survey. *arXiv preprint arXiv:2502.00677* (2025).

[2] Bushra A Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% false positives: A qualitative study of {SOC} analysts' perspectives on security alarms. In *31st USENIX Security Symposium (USENIX Security'22)*. 2783–2800.

[3] Bushra A Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% false positives: A qualitative study of {SOC} analysts' perspectives on security alarms. In *31st USENIX Security Symposium (USENIX Security 22)*. 2783–2800.

[4] Crispin Almodovar, Fariza Sabrina, Sarvnaz Karimi, and Salahuddin Azad. 2022. Can language models help in system security? investigating log anomaly detection using bert. In *Proceedings of the 20th Annual Workshop of the Australasian Language Technology Association*. 139–147.

[5] Anthropic. 2025. Effective Context Engineering for AI Agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents. Accessed: November 1, 2025.

[6] Sidahmed Benabderrahmane, Petko Valtchev, James Cheney, and Talal Rahwan. 2025. APT-LLM: Embedding-Based Anomaly Detection of Cyber Advanced Persistent Threats Using Large Language Models. In *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*. IEEE, 1–6.

[7] Tristan Bilot, Baoxiang Jiang, Zefeng Li, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, and Thomas Pasquier. 2025. Sometimes Simpler is Better: A Comprehensive Analysis of {State-of-the-Art}{Provenance-Based} Intrusion Detection Systems. In *34th USENIX Security Symposium (USENIX Security 25)*. 7193–7212.

[8] CrowdStrike Blog. 2023. Introducing Charlotte AI, CrowdStrike's Generative AI Security Analyst: Ushering in the Future of AI-Powered Cybersecurity. https://www.crowdstrike.com/en-us/blog/crowdstrike-introduces-charlotte-ai-to-deliver-generative-ai-powered-cybersecurity.

[9] Red Canary. 2025. Atomic Red Team. https://www.atomicredteam.io/. [Online; accessed 28-April-2025].

[10] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. Kairos: Practical intrusion detection and investigation using whole-system provenance. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3533–3551.

[11] CMU Information Security Office. 2025. CMU Incident Response Procedure. https://www.cmu.edu/iso/governance/procedures/IRPlan.html.

[12] MITRE Corporation. 2025. MITRE ATT&CK. https://attack.mitre.org/.

[13] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Chenyu Zhao, Shimin Tao, Yilun Liu, Shenglin Zhang, Duoming Lin, Changchang Liu, et al. 2025. LogEval: A comprehensive benchmark suite for LLMs in log analysis. *Empirical Software Engineering* 30, 6 (2025), 173.

[14] Ido Dagan, Oren Glickman, and Bernardo Magnini. 2005. The pascal recognising textual entailment challenge. In *Machine learning challenges workshop*. Springer, 177–190.

[15] DARPA / Five Directions. 2020. Operationally Transparent Cyber (OpTC) Data Release. https://github.com/FiveDirections/OpTC-data.

[16] DARPA I2O. 2020. DARPA Transparent Computing Data Release. https://github.com/darpa-i2o/Transparent-Computing.

[17] Roman Daszczyszak, Dan Ellis, Steve Luke, and Sean Whitley. 2019. TTP-based hunting. *MITRE Corp, McLean VA, Tech. Rep* (2019).

[18] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 847–864.

[19] Feng Dong, Shaofei Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, et al. 2023. Are we there yet? an industrial viewpoint on provenance-based endpoint detection and response tools. In *ACM Conference on Computer and Communications Security*.

[20] Ashish Gehani and Dawood Tariq. 2012. SPADE: support for provenance auditing in distributed environments. In *International Middleware Conference*.

[21] Google Developers. 2025. Prompting Strategies. https://ai.google.dev/gemini-api/docs/prompting-strategies. Accessed: November 1, 2025.

[22] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2024. Logllm: Log-based anomaly detection using large language models. *arXiv preprint arXiv:2411.08561* (2024).

[23] Haixuan Guo, Shuhan Yuan, and Xintao Wu. 2021. Logbert: Log anomaly detection via bert. In *2021 international joint conference on neural networks (IJCNN)*. IEEE, 1–8.

[24] Xueyuan Han, Thomas Pasquier, Adam Bates, James Mickens, and Margo Seltzer. 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats. In *Network and Distributed System Security Symposium (NDSS'20)*. 1–19.

[25] Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X Wang, and Sadid Hasan. 2024. Does prompt formatting have any impact on llm performance? *arXiv preprint arXiv:2411.10541* (2024).

[26] IBM. 2025. IBM QRadar. https://www.ibm.com/products/qradar.

[27] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2023. SoK: History is a Vast Early Warning System: Auditing the Provenance of System Intrusions . In *IEEE Symposium on Security and Privacy (SP)*.

[28] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. {MAGIC}: Detecting advanced persistent threats via masked graph representation learning. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5197–5214.

[29] Gavin Jones, Dimitrios Kasimatis, Nikolaos Pitropakis, Richard Macfarlane, and William J Buchanan. 2025. Analysing the role of LLMs in cybersecurity incident management. *International Journal of Information Security* 24, 6 (2025), 1–14.

[30] Diana Kramer, Lambert Rosique, Ajay Narotam, Elie Bursztein, Patrick Gage Kelley, Kurt Thomas, and Allison Woodruff. 2025. Integrating large language models into security incident response. In *Twenty-First Symposium on Usable Privacy and Security (SOUPS 2025)*. 133–148.

[31] Hwiwon Lee, Ziqi Zhang, Hanxiao Lu, and Lingming Zhang. 2025. SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks. *arXiv preprint arXiv:2506.11791* (2025).

[32] Chenyu Li, Zhengjia Zhu, Jiyan He, and Xiu Zhang. 2025. RedChronos: A Large Language Model-Based Log Analysis System for Insider Threat Detection in Enterprises. *arXiv preprint arXiv:2503.02702* (2025).

[33] Lei Lin, Jiayi Fu, Pengli Liu, Qingyang Li, Yan Gong, Junchen Wan, Fuzheng Zhang, Zhongyuan Wang, Di Zhang, and Kun Gai. 2023. Just ask one more time! self-agreement improves reasoning of language models in (almost) all scenarios. *arXiv preprint arXiv:2311.08154* (2023).

[34] Yilun Liu, Shimin Tao, Weibin Meng, Jingyu Wang, Wenbing Ma, Yuhang Chen, Yanqing Zhao, Hao Yang, and Yanfei Jiang. 2024. Interpretable online log analysis using large language models with prompt strategies. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 35–46.

[35] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. Llmparser: An exploratory study on using large language models for log parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[36] Ashish Bharadwaj Madabhushana. 2021. Configure Linux system auditing with auditd. https://www.redhat.com/en/blog/configure-linux-auditing-auditd.

[37] Microsoft. 2025. Process Monitor v4.01. https://learn.microsoft.com/en-us/sysinternals/downloads/procmon.

[38] Microsoft Learn. 2025. Prompt engineering - Azure AI Foundry. https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/prompt-engineering. Accessed: November 1, 2025.

[39] MITRE ATT&CK. 2025. eCAR - extended Cyber Analytics Repository Model. https://github.com/FiveDirections/OpTC-data/blob/master/ecar.md. [Online; accessed 28-April-2025].

[40] MITRE ATT&CK. 2025. MITRE Cyber Analytics Repository. https://car.mitre.org/. [Online; accessed 28-April-2025].

[41] Alex Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. 2025. Incident Response Recommendations and Considerations for Cybersecurity Risk Management. *NIST SP 800-61r3* (2025).

[42] NVIDIA. 2025. What is Chain of Thought (CoT) Prompting? https://www.nvidia.com/en-us/glossary/cot-prompting/

[43] Thomas Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David Eyers, Margo Seltzer, and Jean Bacon. 2017. Practical whole-system provenance capture. In *Symposium on Cloud Computing*.

[44] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. Flash: A comprehensive approach to intrusion detection via provenance graph representation learning. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3552–3570.

[45] Microsoft Security. 2025. Microsoft Security Copilot. https://www.microsoft.com/en-us/security/business/ai-machine-learning/microsoft-security-copilot.

[46] Minghao Shao, Sofija Jancheska, Meet Udeshi, Brendan Dolan-Gavitt, Kimberly Milner, Boyuan Chen, Max Yin, Siddharth Garg, Prashanth Krishnamurthy, Farshad Khorrami, et al. 2024. Nyu ctf bench: A scalable open-source benchmark dataset for evaluating llms in offensive security. *Advances in Neural Information Processing Systems* 37 (2024), 57472–57498.

[47] Chengyu Song, Linru Ma, Jianming Zheng, Jinzhi Liao, Hongyu Kuang, and Lin Yang. 2024. Audit-llm: Multi-agent collaboration for log-based insider threat detection. *arXiv preprint arXiv:2408.08902* (2024).

[48] U.S. CISA. 2025. CISA Cybersecurity Incident Response. https://www.cisa.gov/topics/cybersecurity-best-practices/organizations-and-cyber-safety/cybersecurity-incident-response.

[49] U.S. Cybersecurity & Infrastructure Security Agency. 2021. Cybersecurity Incident & Vulnerability Response Playbooks. https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks.

[50] U.S. NIST. 2021. Cybersecurity Framework 1.1. https://www.nist.gov/cyberframework/getting-started/online-learning/five-functions.
[51] U.S. NIST. 2025. Special Publication (SP) 800-61 Revision 3. https://csrc.nist.gov/projects/incident-response.
[52] Mark Vero, Niels Mündler, Victor Chibotaru, Veselin Raychev, Maximilian Baader, Nikola Jovanović, Jingxuan He, and Martin Vechev. 2025. BaxBench: Can LLMs generate correct and secure backends? *arXiv preprint arXiv:2502.11844* (2025).
[53] Manfred Vielberth, Fabian Böhm, Ines Fichtinger, and Günther Pernul. 2020. Security Operations Center: A Systematic Study and Open Challenges. *IEEE Access* 8 (2020).
[54] Dawei Wang, Geng Zhou, Xianglong Li, Yu Bai, Li Chen, Ting Qin, Jian Sun, and Dan Li. 2025. The Digital Cybersecurity Expert: How Far Have We Come?. In *2025 IEEE Symposium on Security and Privacy (S&P'25)*. IEEE, 3273–3290.
[55] Lingzhi Wang, Xiangmin Shen, Weijian Li, Zhenyuan Li, R Sekar, Han Liu, and Yan Chen. 2025. Incorporating Gradients to Rules: Towards Lightweight, Adaptive Provenance-based Intrusion Detection. (2025).
[56] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. 2022. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3972–3987.
[57] Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. 2025. CyberGym: Evaluating AI Agents' Cybersecurity Capabilities with Real-World Vulnerabilities at Scale. *arXiv preprint arXiv:2506.02548* (2025).
[58] Wikipedia. 2025. F-score. https://en.wikipedia.org/wiki/F-score.
[59] Yiran Wu, Mauricio Velazco, Andrew Zhao, Manuel Raúl Meléndez Luján, Srisuma Movva, Yogesh K Roy, Quang Nguyen, Roberto Rodriguez, Qingyun Wu, Michael Albada, et al. 2025. Excytin-bench: Evaluating llm agents on cyber threat investigation. *arXiv preprint arXiv:2507.14201* (2025).
[60] Limin Yang, Zhi Chen, Chenkai Wang, Zhenning Zhang, Sushruth Booma, Phuong Cao, Constantin Adam, Alexander Withers, Zbigniew Kalbarczyk, Ravishankar K. Iyer, and Gang Wang. 2024. True Attacks, Attack Attempts, or Benign Triggers? An Empirical Measurement of Network Alerts in a Security Operations Center. In *USENIX Security Symposium (USENIX Security'24)*.
[61] Andy K Zhang, Joey Ji, Celeste Menders, Riya Dulepet, Thomas Qin, Ron Y Wang, Junrong Wu, Kyleen Liao, Jiliang Li, Jinghan Hu, et al. 2025. BountyBench: Dollar Impact of AI Agent Attackers and Defenders on Real-World Cybersecurity Systems. *arXiv preprint arXiv:2505.15216* (2025).
[62] Andy K Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Julian Jasper, et al. [n. d.]. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models. In *The Thirteenth International Conference on Learning Representations (ICLR'25)*.
[63] Jie Zhang, Haoyu Bu, Hui Wen, Yongji Liu, Haiqiang Fei, Rongrong Xi, Lun Li, Yun Yang, Hongsong Zhu, and Dan Meng. 2025. When llms meet cybersecurity: A systematic literature review. *Cybersecurity* 8, 1 (2025), 55.
[64] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
[65] Aoxiao Zhong, Dengyao Mo, Guiyang Liu, Jinbu Liu, Qingda Lu, Qi Zhou, Jiesheng Wu, Quanzheng Li, and Qingsong Wen. 2024. Logparser-llm: Advancing efficient log parsing with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4559–4570.
[66] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, et al. [n. d.]. CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities. In *Forty-second International Conference on Machine Learning (ICML'25)*.

A Dataset Details

A.1 Description of Lab scenarios

Classification (Attack Scenarios).

- Scenario 1: T1563.002 Remote Service Session Hijacking, followed by validation using visudo and appending insecure default which disables the requirement for user must reauth.
- Scenario 2: TA0005 Defense evasion - T1548.003 Abuse Elevation Control Mechanism: Sudo and Sudo Caching
- Scenario 3: T1555.003 Credentials from Password Stores: Credentials from Web Browsers and save the extracted credentials in a file for exfiltration.
- Scenario 4: TA0040 Impact - T1486 Data Encrypted for Impact, followed by deletion of original file and keys
- Scenario 5: A multi-stage attack chain. MITRE Techniques: TA0001 Initial access - T1566.002 Phishing: Spearphishing Link, TA0005 Defense evasion - 1562.001 Impair Defenses: Disable or Modify Tools, TA0002 Execution - T1059.005 Command and Scripting Interpreter: Visual Basic, TA0010 Exfiltration - T1041 Exfiltration Over C2 Channel

Benign Scenarios.

- Benign Scenario 1: A user accesses a remote Ubuntu VM's GUI via VNC (requiring an SSH tunnel) to perform email management (sending/reading emails).
- Benign Scenario 2: Programming activity. A user writes and executes Python scripts using the terminal.
- Benign Scenario 3: A user accesses a cloud-hosted Ubuntu VM's GUI, opens applications (e.g., File Manager, web browser), closes the GUI session once, reopens it, and continues web browsing.
- Benign Scenario 4: Mixed desktop and web activities. A user navigates the file system (terminal), browses the web (Brave), manages emails (Thunderbird), and plays Sudoku.
- Benign Scenario 5: System administration and file management. A user performs routine system maintenance (package updates, storage monitoring, file creation, process inspection) using command-line tools.

Lateral Movement.

- Scenario 1: T1563.002 Remote Service Session Hijacking
- Scenario 2: Credential reuse attack. T1021.001 Remote Services: Remote Desktop Protocol
- Scenario 3: Secure shell lateral access. T1021.004 Remote Services: SSH
- Scenario 4: Centralized management exploitation. T1072 Software Deployment Tools
- Scenario 5: Hash-based authentication bypass. T1550.002 Use Alternate Authentication Material: Pass the Hash

Persistence.

- Scenario 1: SSH authentication bypass. Technique: T1098.004 Account Manipulation: SSH Authorized Keys
- Scenario 2: Local account creation. Technique: T1136.001 Create Account: Local Account
- Scenario 3: Reverse SSH tunnel. Technique: T1133 External Remote Service
- Scenario 4: System startup modification. Technique: T1037.004 Boot or Logon Initialization Scripts: RC Scripts
- Scenario 5: Automated task scheduling. Technique: T1053.003 Scheduled Task/Job: Cron

Data Exfiltration.

- Scenario 1: HTTPS protocol data transfer. T1048.002 Exfiltration Over Alternative Protocol: Exfiltration Over Asymmetric Encrypted Non-C2 Protocol
- Scenario 2: Command and control channel exfiltration. T1041 Exfiltration Over C2 Channel
- Scenario 3: Cloud service data extraction. T1567.002 Exfiltration Over Web Service: Exfiltration to Cloud Storage

- Scenario 4: DNS protocol data transfer. T1048.003 Exfiltration Over Alternative Protocol: Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol
- Scenario 5: Physical media data transfer. T1052.001 Exfiltration Over Physical Medium: Exfiltration over USB

A.2 Description of OpTC scenarios

Classification (Attack Scenarios).

- Scenario 1: Memory credential extraction. Attacker extracts plaintext passwords from system memory and bypasses UAC via Group Policy manipulation.
- Scenario 2: Network reconnaissance via Ping Sweep. Attacker runs automated scripts to discover active hosts across network subnet ranges.
- Scenario 3: UAC bypass and credential search. Attacker exploits Windows Event Viewer for privilege escalation and searches Group Policy Objects for stored credentials.
- Scenario 4: System profiling and process migration. Attacker uses remote access tool modules to discover system/network info (apps, domain, shares) and migrates to critical system processes.
- Scenario 5: Advanced memory credential extraction. Attacker deploys specialized credential extraction tools while maintaining external command and control (C2) communication.

Lateral Movement.

- Scenario 1: Lateral access based on WMI. Attacker uses Windows Management Instrumentation (WMI) to remotely execute commands and establish access on a target system.
- Scenario 2: Lateral access based on administrative WMI. Attacker leverages administrative credentials via WMI to pivot from a compromised system to another network host.
- Scenario 3: Lateral access based on Powershell. Attacker injects a PowerShell script to establish an encrypted communication channel with an external C2.
- Scenario 4: Lateral access based on WMI with domain controller targeting. Attacker uses WMI to pivot from a compromised machine to a high-value domain controller.
- Scenario 5: Lateral access based on an RPC-based network. Attacker performs systematic lateral movement across network segments using Remote Procedure Call connections on administrative ports.

Persistence.

- Scenario 1: T1547.001 Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder.
- Scenario 2: T1546.003 Event Triggered Execution: Windows Management Instrumentation Event Subscription, T1053.005 Scheduled Task/Job: Scheduled Task.
- Scenario 3: T1547.001 Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder.
- Scenario 4: T1133 External Remote Services.

Data Exfiltration.

- Scenario 1: Command and control channel data exfiltration. Attacker compresses the documents directory into a ZIP archive and exfiltrates it to an external C&C server using a network transfer utility.
- Scenario 2: Alternative protocol data exfiltration. Attacker compresses network share files into a multi-gigabyte archive and exfiltrates it to an external server using a custom transfer tool over an RDP session.

B Prompts for Investigation Tasks

B.1 Prompt v1 template

You will be given a set of [Linux / Windows] system logs and need to [SECURITY TASK DEFINITION]. Examine ALL the lines of the audit log input.
Task: [*task description*]
Output Format: [*output format description*]
Input data description **(for Linux raw logs)**: The input audit log data of Linux system obtained directly from Auditd framework that should be used for [task] is as follows: *input data*
OR
Input data description **(for Linux edge representation)**: The audit log data from a Linux system has been processed into an information-flow graph. Each line of input now represents an information flow edge that describes an action between two nodes, and each node corresponds to one of three objects: a process, file and network socket. Each input edge contains the following information: `'(timestamp of action) [unique source node ID] {{key-value pairs of source node attributes separated by commas}} → [unique edge ID] {{key-value pairs of the action's attributes separated by commas}} → [unique destination node ID] {{key-value pairs of destination node separated by commas}}'`.
The input audit log data that should be used for [task] is as follows: [*input data*]

B.2 Prompt v1 for Exfiltration Task

You will be given a set of Windows system logs and need to identify any data exfiltration actions where the user's data is stolen from the machine. Examine ALL the lines of the audit log input.
Task: For each relevant action in the logs that indicates data exfiltration behavior, return the timestamp, the PID of the process that performs exfiltration, full file path of the files that are stolen (to generate the full file path, combine the directory path and file name), the key log lines showing the data exfiltration of the files, and an explanation of the exfiltration activity. Exfiltration actions consist of leaking, copying, or transferring files to external sources, causing data to leave the original system. If you cannot identify any data exfiltration with clear evidence, do not make anything up.
Output format: Output the data exfiltration instances that you identify as a list of JSON, with each JSON object representing an exfiltration instance following this format:
####
{"timestamp": {string, timestamp of the exfiltration action},
"pid": {int, the PID of the process doing the data exfiltration action},
"exfiltrated data": {string, the full file path of the files that are exfiltrated},
"key log lines": {string, list of comma separated log lines showing an exfiltration action and all the data that is exfiltrated by the process as part of this exfiltration action, where every log line should be on a new line},
"explanation": {string, describe all the evidence from the key log lines showing the files that are being exfiltrated. Based on the evidence, write

a claim about whether or not this should be a data exfiltration instance}
}
####
If no data exfiltration activity is found, return an empty JSON object.
[Input data template for raw / edge representation]

B.3 Prompt v2 template

{self.PERSONA}
Goal
{self.goal}
Setup
To accomplish this task, you are given a set of *{self.os_type.value}* audit system logs below.
Input log format
{input_format}
Instructions for reporting *{self.task}* actions
Report your findings, if any, by using one or more JSON objects following the format specified below.
{self.output_description}

```
{{ {output_format}{mitre_output_str} }}
```

If no potential *{self.task}* activity is found, return a single empty JSON object. *{additional_sections_str}*
Data
The input audit log data that should be used for the identification of *{self.task}* is as follows:
{logs}

B.4 Prompt v2 example for exfiltration task

You are a highly-skilled computer security expert with decades of experience in forensic system logs analysis.
Goal
You are tasked with identifying exfiltration activity by a potential attacker, that is, actions that cause user-generated data to leave the system. Look for activities such as external drive shares, USB activity, uploads to websites like Dropbox, iCloud, etc., outgoing data transfer over various protocols like HTTPS, or other types of exfiltration. The user-generated data consist of files and directories that are created by the users of system and the attacker may compress or encrypt them before exfiltration.
Do not report the data that are not user-generated, e.g. system-generated files or temporary files.
You must identify all exfiltration actions that may be used by potential attacker to steal the user-generated data, and assess how suspicious each action is, i.e., likely to be caused by an attacker.
Setup
To accomplish this task, you are given a set of linux audit system logs below.
Input log format
The audit log data from a linux system has been processed into an information-flow graph. Each line of input now represents an information flow edge that describes an action between two nodes, and each node corresponds to one of three objects: a process, file and network socket. Each input edge contains the following information:

```
"(timestamp of action) [unique source node ID]{key-value pairs of source node attributes separated by commas} -> [unique edge ID]{key-value pairs of the action's attributes separated by commas} -> [unique destination node ID]{key-value pairs of destination node separated by commas}"
```

Instructions for reporting exfiltration actions
Report your findings, if any, by using one or more JSON objects following the format specified below.
For each relevant action in the logs that indicates suspicious data exfiltration behavior, return a JSON object following the format specified below.

```
{
    "key log lines": str,  # list of
    comma separated log lines showing
    exfiltration of all the user-generated
    data involved in the data exfiltration
    activity
    "timestamp": str,  # timestamp of
    the exfiltration action.
    "pid": int,  # the PID of the process
    performing the data exfiltration action.
    "exfiltration method": str,  # if the
    user-generated data was  exfiltrated,
    how was it exfiltrated?
    "exfiltrated data": str,  # the
    full file path of the user-generated
    files that are exfiltrated. To
    generate the full file path,
    combine the directory path and
    file name.
    "evidence for": str,  # describe all
    the evidence from the key log lines
    that indicates suspicious data
    exfiltration.
    "evidence against": str,  # describe
    all the evidence from the key
    log lines that indicates either
    not suspicious or not data
    exfiltration.
    "deliberation": str,
    # analyze the evidence above,
    taking into account evidence for
    and against, in order to reach
    a conclusion.
    "verdict": str,  # one of
    ['LOW_SUSPICIOUS', 'MEDIUM_SUSPICIOUS',
    'HIGH_SUSPICIOUS']. Use
    'HIGH_SUSPICIOUS' only when
    the identified behavior is
    highly likely for an attacker to
    create malicious exfiltration.
    Use 'MEDIUM_SUSPICIOUS' when it is
    not clear how feasible it would be
    for an attacker to abuse. Use
    'LOW_SUSPICIOUS' when the activity
    is likely benign, part of regular
    system activity, or generally
    difficult for the attacker to
    use as exfiltration.
}
```

If no potential exfiltration activity is found, return a single empty JSON object.
Data
The input audit log data that should be used for the identification of exfiltration is as follows:
[input data]

Table 11: The effect of log-chunk sizes on Gemini 2.5 Flash and Gemini 2.5 Pro for the raw representation using Prompt v2. (§ 5.3). Across both models, our default chunk size of 1000 lines achieves good performance across the tasks.

Task	Model	Metric	500 lines	1000 lines	2000 lines	4000 lines
Classification	Gem 2.5 Flash	TPR	3/3 (100.00)	3/3 (100.00)	3/3 (100.00)	3/3 (100.00)
		FPR	4/62 (6.45)	2/32 (6.25)	1/17 (5.88)	1/10 (10.00)
L.M.	Gem 2.5 Flash	TPR	1/1 (100.00)	0/1 (0.00)	1/1 (100.00)	1/1 (100.00)
		FPR	36/30425 (0.12)	4/30425 (0.01)	4/30425 (0.01)	4/30425 (0.01)
Persistence	Gem 2.5 Flash	TPR	3/5 (60.00)	3/5 (60.00)	3/5 (60.00)	3/5 (60.00)
		FPR	2/34511 (0.01)	2/34511 (0.01)	2/34511 (0.01)	2/34511 (0.01)
Exfiltration	Gem 2.5 Flash	TPR	2/4 (50.00)	4/4 (100.00)	2/4 (50.00)	2/4 (50.00)
		FPR	15/50267 (0.03)	5/50267 (0.01)	7/50267 (0.01)	9/50267 (0.02)
Classification	Gem 2.5 Pro	TPR	3/3 (100.00)	3/3 (100.00)	3/3 (100.00)	3/3 (100.00)
		FPR	15/62 (24.19)	7/32 (21.88)	5/17 (29.41)	3/10 (30.00)
L.M.	Gem 2.5 Pro	TPR	1/1 (100.00)	1/1 (100.00)	1/1 (100.00)	1/1 (100.00)
		FPR	2/30425 (0.01)	0/30425 (0.00)	1/30425 (0.00)	0/30425 (0.00)
Persistence	Gem 2.5 Pro	TPR	4/5 (80.00)	5/5 (100.00)	5/5 (100.00)	4/5 (80.00)
		FPR	2/34511 (0.01)	0/34511 (0.00)	3/34511 (0.01)	1/34511 (0.00)
Exfiltration	Gem 2.5 Pro	TPR	2/4 (50.00)	4/4 (100.00)	2/4 (50.00)	2/4 (50.00)
		FPR	10/50267 (0.02)	6/50267 (0.01)	11/50267 (0.02)	3/50267 (0.01)

Table 12: Effects of using a more aggressive (true positive optimized) interpretation of a model's output (§ 5.4). The "Δ change" columns shows the absolute change in the metric's value by switching to the more aggressive variant for each model. Green text indicates when the aggressive interpretation improves a metric, and red text indicates that it leads to a worse metric value. Negative values for F1 score and positive values for FPR (higher false positive rates) indicate that using the aggressive interpretation decreases performance.

Task	Metric	Rep	GPT-5 mini, Δ change	Gemini 2.5 Flash, Δ change	Llama 4 Maverick, Δ change	GPT-5, Δ change	Gemini 2.5 Pro, Δ change
classification	F1	edge	-0.20	-0.01	-0.32	-0.33	-0.01
classification	TPR	edge	30.00	0.00	0.00	0.00	0.00
classification	FPR	edge	22.89	7.83	66.27	28.31	8.43
lateral movement	F1	edge	-0.10	-0.11	-0.13	0.21	-0.07
lateral movement	TPR	edge	8.33	0.00	16.67	29.17	0.00
lateral movement	FPR	edge	0.06	0.05	0.22	0.02	0.03
persistence	F1	edge	-0.42	-0.20	-0.20	-0.09	-0.13
persistence	TPR	edge	0.00	0.00	11.11	11.11	0.00
persistence	FPR	edge	0.09	0.08	0.18	0.01	0.08
exfiltration	F1	edge	-0.40	-0.10	-0.32	-0.12	-0.10
exfiltration	TPR	edge	28.57	0.00	42.86	0.00	0.00
exfiltration	FPR	edge	0.06	0.01	0.18	0.00	0.04
classification	F1	raw	-0.14	0.00	-0.20	0.20	-0.03
classification	TPR	raw	0.00	0.00	33.33	33.33	0.00
classification	FPR	raw	3.12	0.00	12.50	0.00	3.12
lateral movement	F1	raw	0.40	0.08	0.20	1.00	-0.33
lateral movement	TPR	raw	100.00	100.00	100.00	100.00	0.00
lateral movement	FPR	raw	0.01	0.06	0.03	0.00	0.00
persistence	F1	raw	-0.22	-0.32	0.20	0.23	-0.41
persistence	TPR	raw	20.00	20.00	60.00	40.00	0.00
persistence	FPR	raw	0.03	0.05	0.02	0.00	0.02
exfiltration	F1	raw	-0.39	-0.17	-0.10	0.13	-0.15
exfiltration	TPR	raw	0.00	0.00	50.00	25.00	0.00
exfiltration	FPR	raw	0.04	0.01	0.05	0.00	0.01